Code in Python

```
import heapq

# Graph: location -> (nearby location, travel time)
graph = {
    'Shop': [('A', 4), ('B', 2)],
    'A': [('Shop', 4), ('B', 1), ('C', 5)],
    'B': [('Shop', 2), ('A', 1), ('C', 8), ('D', 10)],
    'C': [('A', 5), ('B', 8), ('D', 2), ('E', 6)],
    'D': [('B', 10), ('C', 2), ('E', 3)],
    'E': [('C', 6), ('D', 3)]
}


def dijkstra(graph, start):
    distance = {node: float('inf') for node in graph}
    parent = {node: None for node in graph}
    distance[start] = 0

    # Priority queue
    pq = [(0, start)]

    while pq:
        current_distance, current_node = heapq.heappop(pq)

        if current_distance > distance[current_node]:
            continue

        for neighbor, time in graph[current_node]:
            new_distance = current_distance + time

            if new_distance < distance[neighbor]:
                distance[neighbor] = new_distance
                parent[neighbor] = current_node
                heapq.heappush(pq, (new_distance, neighbor))

    return distance, parent


def get_path(parent, node):
    path = []

    while node is not None:
        path.append(node)
        node = parent[node]

    path.reverse()
    return " -> ".join(path)


# Starting point
start = 'Shop'

distance, parent = dijkstra(graph, start)

print("\n--- PIZZA DELIVERY ---")
print("Starting Point:", start)
print("\nShortest Delivery Routes:\n")

for location in distance:
    if location == start:
        continue

    path = get_path(parent, location)

    print("Visit:", location)
    print("Distance from Shop =", distance[location], "minutes")
    print("Route:", path)
    print()

print("--- FINAL RESULT ---")

for location in distance:
    if location != start:
        print("Shop ->", location, "=", distance[location], "minutes")

print("\nAll customers can be reached.")
```